Hash Functions and Types of Hash functions

Last Updated : 10 Mar, 2025

Hash functions are a fundamental concept in computer science and play a crucial role in various applications such as data storage, retrieval, and cryptography. A hash function creates a mapping from an input key to an index in hash table. Below are few examples.

- **Phone numbers as input keys** : Consider a hash table of size 100. A simple example hash function is to consider the last two digits of phone numbers so that we have valid hash table indexes as output. This is mainly taking remainder when input phone number is divided by 100. Please note that taking first two digits of a phone number would not be a good idea for a hash function as there would be many phone number having same first two digits.
- **Lowercase English Strings as Keys** : Consider a hash table of size 100. A simple way to hash the strings would be add their codes (1 for a, 2 for b, ... 26 for z) and take remainder of the sum when divided by 100. This hash function may not be a good idea as strings "ad" and "bc" would have the same hash value. A better idea would be to do weighted sum of characters and then find remainder. Please refer an example string hashing function for details.

Key Properties of Hash Functions

- **Deterministic**: A hash function must consistently produce the same output for the same input.
- **Fixed Output Size**: The output of a hash function should have a fixed size, regardless of the size of the input.
- **Efficiency**: The hash function should be able to process input quickly.
- **Uniformity**: The hash function should distribute the hash values uniformly across the output space to avoid clustering.
- **Pre-image Resistance**: It should be computationally infeasible to reverse the hash function, i.e., to find the original input given a hash value.
- **Collision Resistance**: It should be difficult to find two different inputs that produce the same hash value.

- **Avalanche Effect**: A small change in the input should produce a significantly different hash value.

Applications of Hash Functions

- **Hash Tables**: The most common use of hash functions in DSA is in hash tables, which provide an efficient way to store and retrieve data.
- **Data Integrity**: Hash functions are used to ensure the integrity of data by generating checksums.
- **Cryptography**: In cryptographic applications, hash functions are used to create secure hash algorithms like SHA-256.
- **Data Structures**: Hash functions are utilized in various data structures such as Bloom filters and hash sets.

Types of Hash Functions

There are many hash functions that use numeric or alphanumeric keys. This article focuses on discussing different hash functions:

1. Division Method.
2. Multiplication Method
3. Mid-Square Method
4. Folding Method
5. Cryptographic Hash Functions
6. Universal Hashing
7. Perfect Hashing

Let's begin discussing these methods in detail.

1. Division Method

The division method involves dividing the key by a prime number and using the remainder as the hash value.

> *h(k)=k mod m*
> *Where k is the key and ?m is a prime number.*

Advantages:

- Simple to implement.
- Works well when ?m is a prime number.

Disadvantages:

- Poor distribution if ?m is not chosen wisely.

2. Multiplication Method

In the multiplication method, a constant ?A ($0 < A < 1$) is used to multiply the key. The fractional part of the product is then multiplied by ?m to get the hash value.

> $h(k)=\lfloor m(kA \bmod 1) \rfloor$
> *Where $\lfloor\ \rfloor$ denotes the floor function.*

Advantages:

- Less sensitive to the choice of ?m.

Disadvantages:

- More complex than the division method.

3. Mid-Square Method

In the mid-square method, the key is squared, and the middle digits of the result are taken as the hash value.

Steps:

1. Square the key.
2. Extract the middle digits of the squared value.

Advantages:

- Produces a good distribution of hash values.

Disadvantages:

- May require more computational effort.

4. Folding Method

The folding method involves dividing the key into equal parts, summing the parts, and then taking the modulo with respect to ?*m*.

Steps:

1. Divide the key into parts.
2. Sum the parts.
3. Take the modulo ?*m* of the sum.

Advantages:

- Simple and easy to implement.

Disadvantages:

- Depends on the choice of partitioning scheme.

5. Cryptographic Hash Functions

Cryptographic hash functions are designed to be secure and are used in cryptography. Examples include MD5, SHA-1, and SHA-256.

Characteristics:

- Pre-image resistance.
- Second pre-image resistance.
- Collision resistance.

Advantages:

- High security.

Disadvantages:

- Computationally intensive.

6. Universal Hashing

Universal hashing uses a family of hash functions to minimize the chance of collision for any given set of inputs.

$h(k) = ((a \cdot k + b) \bmod p) \bmod m$

Where a and b are randomly chosen constants, p is a prime number greater than m, and k is the key.

Advantages:

- Reduces the probability of collisions.

Disadvantages:

- Requires more computation and storage.

7. Perfect Hashing

Perfect hashing aims to create a collision-free hash function for a static set of keys. It guarantees that no two keys will hash to the same value.

Types:

- Minimal Perfect Hashing: Ensures that the range of the hash function is equal to the number of keys.
- Non-minimal Perfect Hashing: The range may be larger than the number of keys.

Advantages:

- No collisions.

Disadvantages:

- Complex to construct.

Conclusion

In conclusion, hash functions are very important tools that help store and find data quickly. Knowing the different types of hash functions and how to use them correctly is key to making software work better and more securely. By choosing the right hash function for the job, developers can greatly improve the efficiency and reliability of their systems.

Next Article
Hash Functions and Types of Hash functions

Similar Reads

What are Hash Functions and How to choose a good Hash Function?

What is a Hash Function?A hash function is a function that converts a given large number (such as a phone number) into a smaller, practical integer value. This mapped integer value is used as an index in a...

4 min read

PHP | hash_hmac_algos() Function

The hash_hmac_algos() function is an inbuilt function in PHP that is used to get the list of registered hashing algorithms suitable for the hash_hmac() function.Â Syntax: array hash_hmac_algos(void)...

2 min read

PHP | hash_algos() Function

The hash_algos() function is an inbuilt function in PHP which is used to return a list of registered hashing algorithms. Syntax: array hash_algos(void) Parameter: This function does not accepts any parameter....

2 min read

PHP | hash_copy() Function

The hash_copy() function is an inbuilt function in PHP which is used to get the copy of hashing context. Syntax: hash_copy($context) Parameters: This function accepts single parameter $context which is use...

1 min read

PHP | hash_equals() Function

The hash_equals function() is an inbuilt function in PHP which is used to compares two strings using the same time whether they are equal or not.Syntax: hash_equals($known_str, $usr_str) Parameters: This...

1 min read

Encryption vs Encoding vs Hashing

Pre-Requisite: Encryption, Encoding, Hashing. Encryption, Encoding, and Hahsing are similar kinds of things and have little difference between them. They all are used to change the format of the data or da...

4 min read

What is the difference between Hashing and Hash Tables?

What is Hashing? Hashing refers to the process of generating a fixed-size output from an input of variable size using the mathematical formulas known as hash functions. This technique determines an...

2 min read

Applications, Advantages and Disadvantages of Hash Data Structure

Introduction : Imagine a giant library where every book is stored in a specific shelf, but instead of searching through endless rows of shelves, you have a magical map that tells you exactly which shelf...

7 min read

Applications of Hashing

In this article, we will be discussing of applications of hashing.Hashing provides constant time search, insert and delete operations on average. This is why hashing is one of the most used data structure,...

5 min read

Load Factor and Rehashing

Prerequisites: Hashing Introduction and Collision handling by separate chaining How hashing works: For insertion of a key(K) - value(V) pair into a hash map, 2 steps are required: K is converted into a small...

15+ min read